

Introduction to Python

Module 6 — NumPy

Arrays and fast numerical computing

What Is NumPy?

NumPy is the foundation of numerical computing in Python. Its core object is the **array**: like a list, but built for **fast math** on lots of numbers at once.

```
import numpy as np  
  
arr = np.array([1, 2, 3, 4])  
print(arr)          # [1 2 3 4]  
print(arr.shape)    # (4,) - 4 elements
```

`import numpy as np` is the universal convention — everyone writes `np`. Nearly every data library (including pandas) is built on NumPy underneath.

Why Arrays Beat Lists for Math

The magic: operations apply to the **whole array at once** (“vectorization”) — no loop needed, and far faster.

```
arr = np.array([1, 2, 3, 4])  
  
arr * 2      # [2 4 6 8] - every element doubled  
arr + 10     # [11 12 13 14]  
arr ** 2     # [1 4 9 16]
```

With a plain list, `[1,2,3] * 2` just *repeats* it (`[1,2,3,1,2,3]`). With a NumPy array, `* 2` does real **element-wise math**. That's the key difference.

Useful Array Operations

NumPy has fast built-in summaries — the bread and butter of data analysis.

```
data = np.array([4, 8, 15, 16, 23])  
  
data.sum()      # 66  
data.mean()     # 13.2  
data.max()      # 23  
data.std()      # standard deviation  
  
data[data > 10] # [15 16 23] - filter by condition!
```

That last line — **boolean filtering** — is incredibly powerful: keep only the elements that meet a condition. pandas uses the exact same idea on tables.

Module 6: Takeaways

- ✓ `import numpy as np` — the standard convention
- ✓ A NumPy **array** is like a list, but built for fast numerical math
- ✓ Operations are **vectorized**: `arr * 2` acts on every element
- ✓ Built-in summaries: `.sum()` `.mean()` `.max()` `.std()`
- ✓ **Boolean filtering** `arr[arr > 10]` keeps matching elements

Next: Module 7 — **pandas**, for working with real data tables.